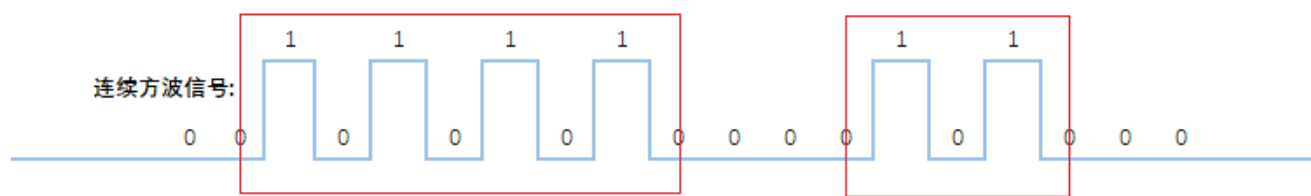


最长方连续方波信号

题目描述

输入一串方波信号，求取最长的完全连续交替方波信号，并将其输出，如果有相同长度的交替方波信号，输出任一即可，方波信号高位用 1 标识，低位用 0 标识，如图：



二进制表示: 001010101010000101000

CSDN @伏城之外

说明：

1. 一个完整的信号一定以 0 开始然后以 0 结尾，即 010 是一个完整信号，但101， 1010， 0101不是
2. 输入的一串方波信号是由一个或多个完整信号组成
3. 两个相邻信号之间可能有 0 个或多个低位，如 0110010， 011000010
4. 同一个信号中可以有连续的高位，如 01110101011110001010，前 14 位是一个具有连续高位的信号
5. 完全连续交替方波是指 10 交替，如 01010 是完全连续交替方波，0110 不是

输入描述

输入信号字符串（长度 ≥ 3 且 ≤ 1024 ）：

0010101010110000101000010

注：输入总是合法的，不用考虑 异常情况

输出描述

输出最长的完全连续交替方波信号串：

01010

若不存在完全连续交替方波信号串，输出 -1。

用例

输入	00101010101100001010010
输出	01010
说明	输入信号串中有三个信号： 0 010101010110(第一个信号段) 00 01010(第二个信号段) 010(第三个信号段) 第一个信号虽然有交替的方波信号段，但出现了11部分的连续高位，不算完全连续交替方波， 在剩下的连续方波信号串中01010最长

题目解析

本题要求“最长的完全交替连续方波信号”，其中形成“完全交替连续方波信号”由两个要点：

- “完全交替连续方波”：完全连续交替方波是指 10 交替
- “信号”：一个完整的信号一定以 0 开始然后以 0 结尾

因此，“完全交替连续方波信号”，可以有两种描述方式：

1. 一个以 0 开头，之后有一个或多个 10 的字符串，用正则表示的话，即为 $^0(10)^+$
2. 开头是一个或多个 01，结尾是 0 的字符串，用正则表示的话，即为 $^(01)^+0^+$

下面源码实现中，我们用了第二种正则，来判断一个字符串是否为“完全交替连续方波信号”

题目中还描述：

输入的一串方波信号是由一个或多个完整信号组成

两个相邻信号之间可能有 0 个或多个低位

因此，当遇到相邻的两个 0 时，就是出现的两个信号（输入总是合法的，不用考虑异常情况）

另外，输入中的信号，不仅有可能是“完全交替信号”，也可能是“不完全交替信号”，如 0110，是一个完整信号，但是是非完全交替信号。

因此，我的解题思路如下，定义一个栈 **stack**，然后遍历输入字符串的字符 c ：

- 如果 $stack$ 为空，则 c 直接压入 $stack$
- 如果 $stack$ 不为空，假设 $stack$ 栈顶字符为 top
 1. 如果 $c == '0'$ ，且 $top == '0'$ ，则说明 c 可能是新信号组成，而 $stack$ 中已保存的字符组成的字符串可能是一个完全交替信号，我们用之前的正则来验证即可，如果验证 OK，则记录此完全交替信号的长度，和本身。完成记录后，清空 $stack$ ，记录新信号的 c 。
 2. 如果 $c == '0'$ ，而 $top == '1'$ ，则 c 直接压入 $stack$
 3. 如果 $c == '1'$ ，则 c 直接压入 $stack$

Java 算法源码

```
1 import java.util.Scanner;  
2 import java.util.regex.Pattern;
```

```
3
4 | public class Main {
5   public static void main(String[] args) {
6       Scanner sc = new Scanner(System.in);
7       System.out.println(getResult(sc.nextLine()));
8   }
9
10  public static String getResult(String s) {
11      Pattern reg = Pattern.compile("^(01)+0$");
12
13      String ans = "-1";
14
15      int maxLen = 0;
16
17      StringBuilder sb = new StringBuilder();
18      for (int i = 0; i < s.length(); i++) {
19          char c = s.charAt(i);
20
21          if (c == '0') {
22              if (sb.length() > 0 && sb.charAt(sb.length() - 1) == '0') {
23                  if (sb.length() > maxLen && reg.matcher(sb.toString()).find()) {
24                      maxLen = sb.length();
25                      ans = sb.toString();
26                  }
27                  sb = new StringBuilder();
28              }
29          }
30
31          sb.append(c);
32      }
33
34      if (sb.length() > maxLen && reg.matcher(sb.toString()).find()) {
35          ans = sb.toString();
36      }
37
38      return ans;
39  }
```

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      console.log(getResult(await readline()));
7  })();
8
9  function getResult(s) {
10     const reg = /^(01)+0$/;
11
12     let ans = "-1";
13
14     let maxLen = 0;
15
16     const stack = [];
17
18     for (let c of s) {
19         if (c == "0") {
20             const len = stack.length;
21
22             if (len > 0 && stack.at(-1) == "0") {
23                 const str = stack.join("");
24                 if (len > maxLen && reg.test(str)) {
25                     maxLen = len;
26                     ans = str;
27                 }
28
29                 stack.length = 0;
30             }
31         }
```

[运行](#)

```

32 |
33 |     stack.push(c);
34 | }
35 |
36 | const str = stack.join("");
37 | if (str.length > maxLen && reg.test(str)) {
38 |     ans = str;
39 | }
40 |
41 | return ans;
42 | }

```

Python算法源码

```

1  import re
2
3  # 输入获取
4  s = input()
5
6
7  # 算法入口
8  def getResult():
9      reg = re.compile(r"^(01)+0$")
10
11      maxLen = 0
12      ans = "-1"
13
14      stack = []
15
16      for c in s:
17          if c == "0":
18              if len(stack) > 0 and stack[-1] == "0":
19                  sub = "".join(stack)
20                  if len(sub) > maxLen and reg.match(sub):

```

```

21         maxlen = len(stack)
22         ans = sub
23         stack.clear()
24         stack.append(c)
25
26     sub = "".join(stack)
27     if len(sub) > maxlen and reg.match(sub):
28         ans = sub
29
30     return ans
31
32
33 # 输出打印
34 print(getResult())

```

C算法源码

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdbool.h>
4
5  #define MAX_LEN 1025
6
7  bool check(char *s, int len) {
8      if(len < 3) return false;
9
10     if (s[0] != '0') return false;
11
12     for (int i = 1; i < strlen(s); i += 2) {
13         if (s[i] != '1' || s[i + 1] != '0') return false;
14     }
15
16     return true;
17 }
18

```

```

19 | int main() {
20 |     char s[MAX_LEN] = {'\0'};
21 |     scanf("%s", s);
22 |
23 |     char ans[MAX_LEN] = "-1";
24 |
25 |     int maxLen = 0;
26 |
27 |     char stack[MAX_LEN] = {'\0'};
28 |     int stack_size = 0;
29 |     for (int i = 0; i < strlen(s); i++) {
30 |         char c = s[i];
31 |
32 |         if (c == '0') {
33 |             if (stack_size > 0 && stack[stack_size - 1] == '0') {
34 |                 if (stack_size > maxLen && check(stack, stack_size)) {
35 |                     maxLen = stack_size;
36 |                     strcpy(ans, stack);
37 |                 }
38 |                 memset(stack, '\0', MAX_LEN);
39 |                 stack_size = 0;
40 |             }
41 |         }
42 |
43 |         stack[stack_size++] = c;
44 |     }
45 |
46 |     if (stack_size > maxLen && check(stack, stack_size)) {
47 |         strcpy(ans, stack);
48 |     }
49 |
50 |     puts(ans);
51 |
52 |     return 0;
53 | }

```


C++ 算法源码

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      string s;
7      cin >> s;
8
9      regex pattern("^(01)+0$");
10
11     string ans = "-1";
12
13     string stack;
14     int maxLen = 0;
15
16     for (const auto &c: s) {
17         if (c == '0') {
18             int len = stack.length();
19
20             if (len > 0 && stack[len - 1] == '0') {
21                 if (len > maxLen && regex_search(stack, pattern)) {
22                     maxLen = len;
23                     ans = stack;
24                 }
25
26                 stack = "";
27             }
28         }
29
30         stack += c;
31     }
32
33     if (stack.length() > maxLen && regex_search(stack, pattern)) {
34         ans = stack;
35     }
```

```
36 |  
37 |     cout << ans << endl;  
38 |  
39 |     return 0;  
40 | }
```